GCD Data and Comic Metadata Containers in Codex

Status: Design handoff; no Codex or library files were modified. **Prepared:** 2026-11-09 **Purpose:** Transfer findings from a read-only scan of `../codex` to CB-MANAGE.

Executive summary

Codex already has substantial support for reading, modeling, editing, and writing comic archive metadata through its `comicbox` dependency. Its current tagging surfaces expose both MetronInfo and ComicInfo formats. A local GCD dataset enrichment workflow therefore appears feasible as an external, carefully staged tool or as a proposed Codex feature. The repository scan does not establish that Codex already imports GCD data or has a GCD-driven metadata update feature.

Keep three responsibilities distinct:

1. **GCD snapshot intake:** acquire and refresh a permitted local GCD data copy, record version/date/provenance, and transform it into a queryable local index.
2. **Record matching and review:** match a Codex comic to a GCD issue/series record, expose candidate evidence and confidence, and require review of ambiguous matches.
3. **Container update:** make a narrowly scoped change to `MetronInfo.xml`, `ComicInfo.xml`, or both inside a supported archive, preserving existing information and recording a before/after audit.

Do not couple GCD downloading, matching, and archive writes into one opaque batch operation. Treat GCD as a candidate source until its terms, attribution, data scope, refresh approach, and exact fields are reviewed and recorded in CB-MANAGE truth.

Codex scan findings

The scan covered the local `../codex` checkout. Relevant implementation areas:

- `codex/choices/tagging.py` imports `ComicInfoTransform` and `MetronInfoTransform` from `comicbox`, and builds editor format choices from those transforms.
- `codex/librarian/scribe/tasks.py` defines bulk tag-write tasks, including selected formats, patches, key deletion, and rename options.
- `codex/librarian/scribe/tag_writer.py` delegates archive writes to `comicbox.write.bulk_write`, collects final paths, reports per-file errors, and re-imports metadata after a write. It contains explicit safeguards around archive conversion/renaming and write configuration.
- `codex/librarian/scribe/importer/read/` extracts, aggregates, and maps archive metadata into Codex's database models.
- `codex/README.md` documents embedded ComicInfo support via `comicbox`.
- Existing tests cover parts of tagging, archive write behavior, metadata import, and file conversion. They are useful design references, but do not prove compatibility with a proposed GCD enrichment pipeline.

This is a source inspection, not a runtime test or compatibility guarantee. Before implementation, inspect the exact installed/locked `comicbox` version and its MetronInfo/ComicInfo transform and write APIs. The current Codex code already uses these APIs, so extending its supported path may be safer than inventing a second archive-rewrite implementation.

GCD and source-of-truth status

CB-MANAGE `truth/EXTERNAL/manifest.md` lists the Grand Comics Database at `https://www.comics.org/` as candidate `CB-EXT-010`. The Metron Project is candidate `CB-EXT-008`. The manifest explicitly says these URLs have not been verified or promoted to authoritative status. `truth/sites.md` also contains CLU and Pullbox links, but those entries are not yet incorporated into the formal manifest/index as reviewed sources.

Before depending on a local GCD dataset, verify and record:

- permitted dataset acquisition and reuse, attribution, redistribution, and derived-data requirements;

- which entities and fields the available dataset actually contains;
- stable keys, relationship tables, language/credit conventions, and update cadence;
- snapshot date, source version, import command, transformation code version, and checksums;
- whether local changes can be refreshed incrementally or require a full re-import.

Do not infer GCD licensing or database completeness from its public website. Use an approved source record with exact documentation URLs and review date.

Proposed data flow

```
Approved GCD snapshot
       |
       v
Versioned local staging/index -----> provenance and schema checks
       |                                        |
       v                                        v
Candidate lookup by identifiers, title, issue, year, publisher, volume
       |
       v
Match report with evidence, confidence, and field-level proposed chang
       |
       v
Human approval / explicit batch scope
       |
       v
Archive backup -> targeted XML patch -> validate -> atomic replace
       |
       v
Re-read with comicbox/Codex -> compare -> append audit result
```

Prefer strong identifiers when present, but do not assume that a GCD identifier is already represented in every archive or in Codex's current identifier mapping. Fall back to composite evidence only as a candidate match. A title and issue number alone are not sufficient for silent bulk

writes. Keep unmatched, ambiguous, and conflicting matches in a review queue.

Metadata-container behavior and risks

- `MetronInfo.xml` and `ComicInfo.xml` are distinct schemas and may represent the same concept differently. Map each GCD field separately for each target format; do not copy XML elements by name or assume one format is a superset.
- A container may have one format, both, or neither. Define whether the tool updates only existing containers, creates a selected container, or writes both. Default to an explicit per-format selection.
- Read-modify-write must preserve unrelated fields, unknown extension fields, namespaces, ordering where practical, and original archive members. Avoid replacing a full XML document from a partial GCD record.
- Define precedence for conflicting values among existing XML, GCD, archive filename, and Codex's database. GCD data should not automatically overwrite user-curated values.
- Handle encoding, XML entities, namespaces, duplicate XML members, malformed XML, large archives, unsupported archive formats, and interrupted writes.
- Preserve the original archive or a verified backup until post-write validation succeeds. Write to a temporary destination and replace atomically when supported; verify archive readability and the edited XML before considering the operation successful.
- Account for Codex's filesystem watcher, archive mtime checks, metadata re-import, renaming, conversion behavior, and final-path changes. Avoid competing with an active Codex tagging/import job.
- Record source snapshot/version, match evidence, fields changed, old/new values, selected format, archive identity, result, and rollback location. Keep private library paths and personal metadata out of public logs.

Proposed CB-MANAGE skillset

The README now defines a planned **comic metadata container skillset** for reading, analyzing, and updating `MetronInfo.xml` and `ComicInfo.xml`. It is a scope definition, not an installed or validated executable skill. Suggested future bundle: `.agents/skills/comic-metadata-containers/SKILL.md`, with references for format mappings, GCD source provenance, dry-run reports, and rollback/verification procedures.

Required operating modes:

1. **Inspect:** identify archives and contained metadata files; parse without mutation; report malformed, absent, duplicate, and conflicting metadata.
2. **Analyze:** compare existing fields with a selected GCD match; explain match evidence, confidence, source provenance, and field differences.
3. **Plan:** emit a dry-run patch showing exact archive, format, XML paths, current values, proposed values, and policy/approval requirements.
4. **Update:** only after explicit scope approval; back up, patch selected elements, validate, replace, re-read, and record an audit/rollback path.

Default to inspect/analyze/dry-run. Never scan an unspecified library root, silently select a low-confidence match, overwrite conflicting curated data, or perform broad writes without a user-approved batch scope. Support a small-fixture test mode before any real library run.

Suggested implementation and validation sequence

1. Promote and document the GCD and Metron sources only after checking their exact data-use terms and technical documentation; keep other URLs candidate.
2. Inspect the locked `comicbox` package APIs and Codex's existing writer, importer, watcher, and rename lifecycle in the target Codex revision.
3. Define the local GCD schema and reproducible snapshot import, provenance, and refresh process.
4. Define identifier precedence, matching keys, ambiguity thresholds, and field-level conflict rules with representative comics.
5. Build inspect/analyze/dry-run first; test malformed archives, both container formats, archives with both formats, missing XML, conflicting

values, duplicate members, and interrupted writes.

6. Validate each proposed patch against the format transform, then re-read the written archive with the same `comicbox` path Codex uses.
7. Test watcher/import/rename integration and rollback on copies before enabling updates on the live library.
8. Start with a small explicitly approved batch and compare before/after reports; retain backups until the results are accepted.

Scope boundary

This handoff does not modify `../codex`, download GCD, access a comic library, or write any archive metadata. It records what the repository scan found and what needs validation before implementation.